

tmux Mastery Guide

The Mental Model

tmux has a strict three-level hierarchy:

```
Session
├── Windows (like tabs)
│   └── Panes (splits within a window)
```

A **session** is the top-level container. It lives on the tmux server and survives SSH disconnection, laptop closure, terminal crashes — everything. This is the entire reason tmux exists for remote work. Your training run on Oscar keeps going even when your SSH drops.

A **window** is a full-screen workspace within a session. Think of it like a browser tab. Each window is independent and can have its own pane layout.

A **pane** is a split within a window. Multiple shells running side by side inside one window.

tmux runs as a **client-server architecture**. When you type `tmux`, you start both a server (which holds all sessions in memory) and a client (your terminal). The server lives until you reboot or kill it explicitly. This is why sessions survive disconnection — the server never stopped.

The **prefix key** (`(C-b)` by default) is how you signal tmux that the next keypress is a tmux command, not input to your shell. Press `(C-b)`, release, then press the command key.

Sessions

From the shell (outside tmux)

Command	What it does
<code>tmux</code>	New session, auto-numbered
<code>tmux new -s name</code>	New named session
<code>tmux new -s name -d</code>	New session, detached (no terminal attached — runs in background)
<code>tmux ls</code>	List all sessions

Command	What it does
<code>tmux attach</code>	Attach to most recent session
<code>tmux attach -t name</code>	Attach to named session
<code>tmux attach -t name -d</code>	Attach, forcing any other client off that session
<code>tmux kill-session -t name</code>	Kill a specific session
<code>tmux kill-server</code>	Kill everything — all sessions, the server
<code>tmux has-session -t name</code>	Check if session exists (exit code 0 = yes, 1 = no)
<code>tmux rename-session -t old new</code>	Rename a session

Inside tmux (keybinds)

Keybind	What it does
<code>C-b d</code>	Detach from current session
<code>C-b D</code>	Choose which client to detach (when multiple terminals are attached)
<code>C-b \$</code>	Rename current session
<code>C-b s</code>	Interactive session/window tree — navigate with arrows, Enter to switch
<code>C-b (</code>	Switch to previous session
<code>C-b)</code>	Switch to next session
<code>C-b L</code>	Toggle to last used session

Windows

Keybind	What it does
<code>C-b c</code>	Create new window
<code>C-b ,</code>	Rename current window

Keybind	What it does
<code>C-b n</code>	Next window
<code>C-b p</code>	Previous window
<code>C-b l</code>	Last (most recently used) window
<code>C-b 0-9</code>	Jump to window by number
<code>C-b w</code>	Interactive window list
<code>C-b &</code>	Kill current window (prompts for confirmation)
<code>C-b .</code>	Move window to a different index number
<code>C-b f</code>	Find and switch to window by name

Panes

Splitting

Keybind	What it does
<code>C-b %</code>	Split left/right (vertical divider)
<code>C-b "</code>	Split top/bottom (horizontal divider)

Navigating

Keybind	What it does
<code>C-b arrow</code>	Move to pane in that direction
<code>C-b q</code>	Flash pane numbers — press the number to jump to that pane
<code>C-b ;</code>	Toggle between current and last active pane
<code>C-b o</code>	Cycle through panes in order

Resizing

Keybind	What it does
<code>C-b C-arrow</code>	Resize pane 1 cell in that direction (hold Ctrl, tap arrow repeatedly)
<code>C-b M-arrow</code>	Resize in larger steps (M = Alt)

From command mode: `:resize-pane -D 10` (Down 10 cells), `-U`, `-L`, `-R`

Managing

Keybind	What it does
<code>C-b x</code>	Kill current pane (prompts for confirmation)
<code>C-b z</code>	Zoom pane to full screen — press again to unzoom
<code>C-b !</code>	Break pane out into its own window
<code>C-b {</code>	Swap current pane with the previous one
<code>C-b }</code>	Swap current pane with the next one
<code>C-b space</code>	Cycle through layouts: even-horizontal, even-vertical, main-horizontal, main-vertical, tiled

Copy Mode

This is what the tutorials skipped. When your training job dumps 500 lines of output and you need to scroll back and find an error — this is what you use.

Entering and Exiting

Keybind	What it does
<code>C-b [</code>	Enter copy mode
<code>C-b PgUp</code>	Enter copy mode AND immediately scroll up one page
<code>q</code> or <code>Escape</code>	Exit copy mode

With `set -g mouse on`, scrolling the mouse wheel automatically enters copy mode. This is the main reason mouse mode is useful.

Navigation Inside Copy Mode

Key	What it does
Arrow keys	Move cursor one line/character
<code>PgUp</code> / <code>PgDn</code>	Scroll by full page
<code>g</code>	Jump to top of scrollback buffer
<code>G</code>	Jump to bottom (back to present output)
<code>C-b</code>	Scroll up half page
<code>C-f</code>	Scroll down half page
<code>H</code>	Top of currently visible area
<code>M</code>	Middle of currently visible area
<code>L</code>	Bottom of currently visible area

Searching in Copy Mode

Key	What it does
<code>/</code>	Search forward (downward)
<code>?</code>	Search backward (upward)
<code>n</code>	Jump to next match
<code>N</code>	Jump to previous match

Copying Text

Key	What it does
<code>Space</code>	Begin selection

Key	What it does
Arrow keys	Extend selection
<code>Enter</code>	Copy selection, exit copy mode
<code>C-b]</code>	Paste copied text

Scrollbar Buffer Size

By default tmux keeps only 2000 lines. For ML training logs you want much more. Add this to `~/.tmux.conf`:

```
set -g history-limit 50000
```

Command Mode

`C-b :` opens a command prompt at the bottom of the screen. You can run any tmux command directly without touching your config file. Useful for testing options on the fly.

```
:set -g mouse on           # toggle an option without editing config
:source-file ~/.tmux.conf  # reload config
:split-window -h           # split pane (same as C-b %)
:kill-session              # kill current session
:new-window                 # new window
:resize-pane -D 10         # resize down 10 cells
:set -g history-limit 50000 # increase scrollbar
:swap-window -s 2 -t 1     # swap window 2 and window 1
```

`C-b ?` lists ALL keybindings. Searchable with `/`, press `q` to exit. Use this whenever you forget a command.

Scripting tmux

This is what turns tmux from an interactive tool into an automation tool — important for Oscar workflows.

Core Commands for Scripting

Command	What it does
<code>tmux new-session -d -s name</code>	Create a detached session (no terminal attached)
<code>tmux new-session -d -s name 'cmd'</code>	Create session and run a command in it immediately
<code>tmux send-keys -t target 'cmd'</code> <code>Enter</code>	Send keystrokes to a pane as if typed
<code>tmux new-window -t session -n name</code>	Add a named window to a session
<code>tmux select-window -t session:1</code>	Switch to window 1 in a session
<code>tmux select-pane -t target</code>	Select a specific pane
<code>tmux list-windows -t session</code>	List windows in a session
<code>tmux list-panes -t session:window</code>	List panes in a window
<code>tmux has-session -t name</code>	Check if session exists (use in scripts)
<code>tmux display-message -p '#S'</code>	Print current session name

The Target Syntax: `session:window.pane`

The `(-t)` flag accepts targets in the form `session:window.pane`. Examples:

```
training          → session named "training"
training:0        → window 0 of session "training"
training:0.1      → pane 1 of window 0 of session "training"
training:logs     → window named "logs" of session "training"
```

A Real Oscar Setup Script

```
bash
```

```
#!/bin/bash
SESSION="ml_work"

if ! tmux has-session -t $SESSION 2>/dev/null; then
    tmux new-session -d -s $SESSION -n editor      # window 0: code editor
    tmux new-window -t $SESSION -n training      # window 1: run jobs
    tmux new-window -t $SESSION -n logs          # window 2: watch logs

    # Start watching logs immediately in window 2
    tmux send-keys -t $SESSION:logs 'tail -f ~/logs/train.log' Enter

    # Start at window 0
    tmux select-window -t $SESSION:editor
fi

tmux attach -t $SESSION
```

Save this as `~/workspace.sh`, `chmod +x` it, and run it on login. You always land in your workspace exactly as configured, whether it already existed or you're creating it fresh.

Nested Sessions (Oscar-specific)

When you SSH into Oscar and run tmux there, you have two nested tmux sessions. Your local tmux on the outside, Oscar's tmux on the inside.

The problem: `C-b` gets captured by your local tmux. Commands never reach Oscar's tmux.

Solution 1 — Press prefix twice: `C-b C-b` sends one `C-b` through to the inner session. So to split a pane in Oscar's tmux: `C-b C-b %`

Solution 2 — Different prefix on Oscar (recommended for regular use): In your `~/tmux.conf` on Oscar:

```
set -g prefix C-a
bind C-a send-prefix
unbind C-b
```

Now locally you use `C-b`, on Oscar you use `C-a`. Zero conflicts. This is the standard approach.

The Oscar/SSH Workflow

```
bash

# --- On Oscar for the first time ---
ssh oscar
tmux new -s ml_project

# Set up your workspace inside
# C-b c to create windows, C-b , to name them
# Start your training job
python train.py

# Detach safely – job keeps running on the server
C-b d

# --- SSH connection drops, you close your laptop, anything ---
# Your session is completely fine on the server

# --- Coming back later ---
ssh oscar
tmux ls                # see all running sessions
tmux attach -t ml_project # pick up exactly where you left

# --- Safe disconnect pattern when done for the day ---
C-b d                  # detach, session survives
# vs
tmux kill-session -t ml_project # actually end the session
```

Useful one-liner for your Oscar `.bashrc`:

```
bash

alias tm='tmux attach 2>/dev/null || tmux new -s main'
```

Type `tm` on login — attaches to existing session if one exists, creates a new one called "main" if not.

Config Reference

```
bash
```

```
# ~/.tmux.conf

# What you currently have
set -g mouse on

# Add this – essential for ML logs
set -g history-limit 50000

# Reload config with C-b r
bind r source-file ~/.tmux.conf

# ---- Understanding the syntax ----

# set -g = set-option -g (global, applies to all sessions)
# setw -g = set-window-option -g (applies to windows)
# bind = bind-key (requires prefix key first)
# bind -n = bind without needing prefix (direct keybind)
# bind -r = repeatable (can press key multiple times after one prefix press)
```

Applying config changes:

- From shell: `tmux source ~/.tmux.conf`
- From inside tmux: `C-b :source-file ~/.tmux.conf`
- Or use the `bind r` shortcut above once it's set

Quick Reference Card

Things that become motor memory through real use:

What you want	How
Detach from session	<code>C-b d</code>
Scroll back through output	<code>C-b [</code> then PgUp
Return to present	<code>G</code> in copy mode, or <code>q</code>
Split screen side by side	<code>C-b %</code>
Split screen top/bottom	<code>C-b "</code>
Move between panes	<code>C-b arrow</code>
Zoom a pane	<code>C-b z</code>
New window	<code>C-b c</code>
Switch windows	<code>C-b n</code> / <code>C-b p</code> / <code>C-b number</code>
See all sessions	<code>C-b s</code>
All keybindings	<code>C-b ?</code>
Run any tmux command	<code>C-b :</code>

What You Don't Need to Memorize

Everything above `C-b ?` will find for you. The scripting commands you'll look up until they stick through actual Oscar use. The only things worth having in your fingers before Oscar:

1. Session management: `tmux new -s`, `tmux attach -t`, `tmux ls`, `C-b d`
2. Copy mode: `C-b [`, PgUp, `g`/`G`, `q`
3. Pane navigation: `C-b arrow`, `C-b z`
4. Window switching: `C-b c`, `C-b n/p/number`